

CodeTober 2026 - Roadmap Data Engineering

24 défis progressifs : des fondations Python/data vers une pipeline complète, incrémentale, robuste, observable et testée.

Principe de notation

4/5 : l'implémentation satisfait correctement toutes les exigences fonctionnelles et contraintes obligatoires de l'énoncé.

5/5 : l'énoncé ajoute dès le départ une contrainte d'ingénierie explicite (architecture, performance, robustesse, mémoire, testabilité, etc.). La propriété attendue est donnée, mais pas la solution technique : le choix du mécanisme fait partie du défi.

Les CSV peuvent rester des jeux de données pratiques, mais ils ne sont plus l'objectif pédagogique. À partir de J7, les défis doivent progressivement construire de vrais réflexes de Data Engineering : état, incrémental, idempotence, persistance, SQL, qualité, orchestration, résilience et observabilité.

J1 à J6 - Fondations déjà acquises

J1 - dirty_csv

Validation et normalisation. Nettoyer un CSV de transactions, valider les champs, rejeter les lignes incorrectes et normaliser les valeurs sans charger le fichier entier en mémoire. Introduction aux tests unitaires avec pytest.

J2 - chunk_me_if_you_can

Agrégations et volumétrie. Analyser des logs API en une passe : compteurs de statuts, statistiques par service, moyenne des temps de réponse et top-K des requêtes les plus lentes. Travail sur les structures de données et la mémoire.

J3 - schema_says_no

Validation conditionnelle et JSON. Valider des événements dont les règles dépendent du type d'événement, notamment des champs optionnels/obligatoires et des metadata JSON. Mise en pratique du TDD sur les fonctions de validation.

J4 - join_me_if_you_can

Jointure et lookup. Joindre deux datasets en choisissant intelligemment ce qui doit être chargé en mémoire. Utiliser un dictionnaire indexé par clé métier pour obtenir des recherches efficaces et raisonner sur la complexité.

J5 - divide_and_conquer

Partitionnement et gestion des ressources. Router un flux de commandes vers des partitions date/région, dédupliquer par partition et écrire au fil de l'eau. Le prolongement a introduit la gestion dynamique des fichiers, ExitStack, les limites de descripteurs et un cache LRU borné.

J6 - Checkpoint

État persistant et reprise. Conserver la version la plus récente de chaque événement. Pour le 5/5 : permettre une reprise après interruption sans retraiter tout le fichier, avec un état durable et cohérent même en cas de crash.

J7 à J12 - De scripts à pipeline fiable

J7 - Watermark

Ingestion incrémentale. Ne traiter que les données nouvelles depuis le dernier run. Définir et maintenir un watermark, gérer les relances et commencer à raisonner systématiquement en termes d'idempotence.

J8 - State of Affairs

Persistance et transactions. Sortir l'état important de la mémoire du processus et le rendre durable. Introduire une base locale et réfléchir aux frontières transactionnelles : quelles modifications doivent réussir ou échouer ensemble ?

J9 - Upsert or Shut Up

SQL, clés métier et rejouabilité. Charger des données dans une table cible sans dupliquer les enregistrements lors d'une relance. Distinguer insert, update et upsert, choisir les clés pertinentes et rendre le chargement rejouable.

J10 - Quality Control

Data quality gates. Définir plusieurs règles de qualité, mesurer les anomalies et décider si un lot peut continuer. Séparer données valides, données en quarantaine et métriques de qualité.

J11 - Pipe Dreams

Pipeline Extract - Transform - Load. Assembler plusieurs étapes dans une vraie chaîne de traitement. Séparer suffisamment les responsabilités pour tester et faire évoluer extraction, transformation et chargement indépendamment.

J12 - Schema Evolution

Évolution des schémas. Faire survivre la pipeline à l'arrivée, la disparition ou l'évolution de colonnes. Définir ce qui est compatible, ce qui doit être rejeté et comment faire évoluer les transformations.

J13 à J18 - Résilience, orchestration et SQL

J13 - Retry Me Maybe

Résilience. Distinguer erreur permanente et erreur transitoire. Ajouter une stratégie de retry raisonnable, éviter les boucles infinies et préserver l'idempotence lorsqu'une étape est rejouée.

J14 - DAG Nabbit

Dépendances et orchestration. Modéliser des tâches et leurs dépendances sous forme de graphe. Déterminer un ordre d'exécution valide, empêcher l'exécution d'une tâche dont les prérequis ont échoué et détecter les dépendances impossibles.

J15 - Parallel Universe

Concurrence. Exécuter certaines tâches indépendantes en parallèle. Mesurer le gain réel, identifier les ressources partagées et éviter que la concurrence introduise des incohérences.

J16 - Queue-rious

Producer / Consumer. Découpler production et traitement grâce à une file de travail. Gérer la vitesse différente des producteurs/consommateurs, la fin du traitement et les erreurs partielles.

J17 - Slowly but Surely

Historisation / SCD. Conserver l'historique d'une dimension au lieu d'écraser son état précédent. Implémenter une logique de Slowly Changing Dimension et raisonner sur les périodes de validité.

J18 - Window Shopping

SQL analytique. Utiliser des fonctions fenêtre pour ranking, cumul, comparaison avec la ligne précédente et agrégations par groupe sans perdre le niveau de détail des lignes.

J19 à J24 - Vers une vraie plateforme data

J19 - Observe This

Observabilité. Donner à chaque exécution une identité et produire des logs/métriques exploitables : durée, volumes, erreurs, étapes exécutées et statut final. L'objectif est de diagnostiquer un run sans relire le code.

J20 - Configurable

Pipeline pilotée par configuration. Faire varier sources, destinations et comportements sans modifier le code métier. Valider la configuration et éviter qu'elle devienne un second langage de programmation incontrôlable.

J21 - Duck Season

Analyse locale avec DuckDB. Interroger efficacement des datasets depuis SQL sans tout charger dans des structures Python. Comparer le rôle du moteur analytique avec les transformations Python déjà pratiquées.

J22 - Column Goals

Parquet et stockage colonne. Passer du CSV à un format colonne, comprendre types, compression et partitionnement physique. Concevoir les données pour éviter de lire ce qui n'est pas nécessaire.

J23 - Orchestrate This

Mini-orchestrateur. Combiner DAG, état, retries, reporting et observabilité. Une exécution doit connaître les tâches prêtes, réussies, échouées ou à rejouer et conserver un historique cohérent.

J24 - Final Pipeline

Final boss. Construire une pipeline complète intégrant les briques apprises : ingestion incrémentale, validation, transformation, persistance, idempotence, reprise, qualité, orchestration, logs et tests.

Fil conducteur

La progression n'est pas pensée comme 24 scripts indépendants. À mesure que les jours avancent, les notions précédentes doivent redevenir utiles : une pipeline incrémentale devra aussi être idempotente ; une étape parallèle devra rester observable ; un retry devra respecter les transactions ; une orchestration devra connaître l'état de ses tâches. L'objectif final est de développer des réflexes transférables à un environnement professionnel de Data Engineering, pas de multiplier les exercices de manipulation de CSV.